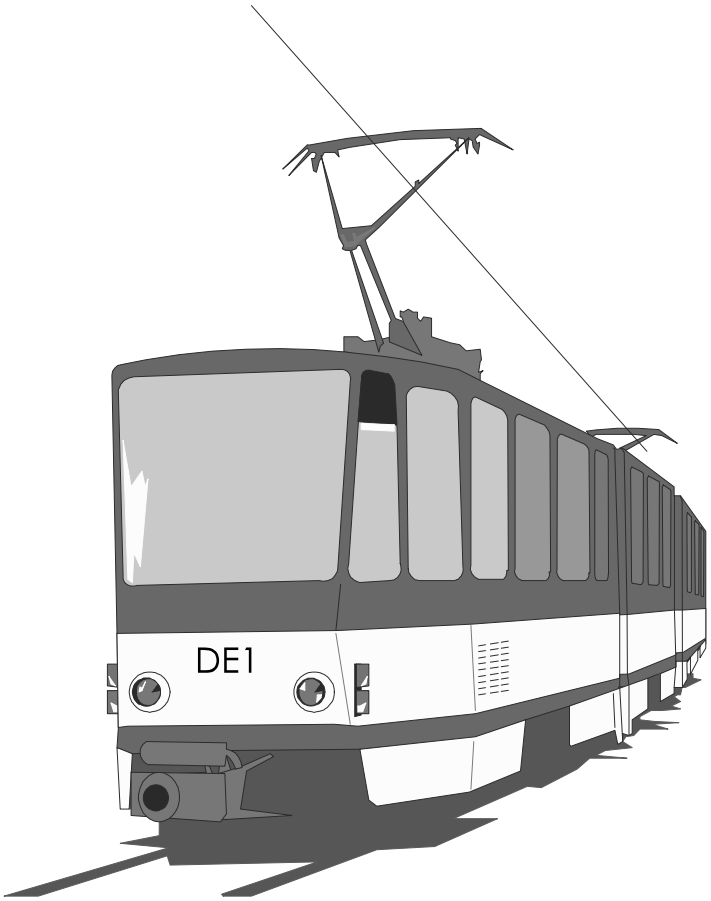
CHAPTER 8

State Machine Design: The Electric Train Controller



8 State Machine Design: The Electric Train Controller

8.1 The Train Control Problem

The track layout of a small electric train system is shown in Figure 8.1. Two trains, we'll call A and B, run on the tracks, hopefully without colliding. To avoid collisions, the trains require a safety controller that allows trains to move in and out of intersections without mishap.

For safe operation, only one train at a time can be present on any given track segment. The track layout seen in Figure 8.1 is divided into four track segments. Each track segment has sensors that are used to detect trains at the entry and exit points.

In Figure 8.1, there are two Trains A and B. As an example, assume Train A always runs on the outer track loop and Train B on the inner track loop. Assume for a moment that Train A has just passed Sensor 4 and is near Switch 3 moving counterclockwise. Let's also assume that Train B is moving counterclockwise and approaching Sensor 2. Since Train B is entering the common track (Track 2), Train A must be stopped when it reaches Sensor 1, and must wait until Train B has passed Sensor 3 (i.e., Train B is out of the common track). At this point, the track switches should switch for Train A, Train A will be allowed to enter Track 2, and Train B will continue moving toward Sensor 2.

The controller is a state machine that uses the sensors as inputs. The controller's outputs control the direction of the trains and the position of the switches. However, the state machine does not control the speed of the train. This means that the system controller must function correctly independent of the speed of the two trains.

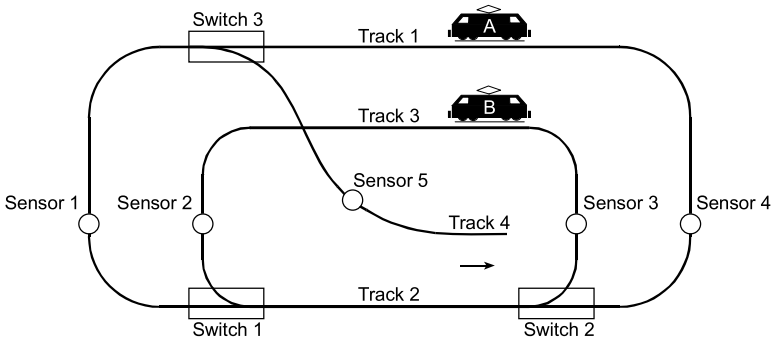


Figure 8.1 Track layout with input sensors and output switches and output tracks.

An FPGA-based "virtual" train simulation will be used that emulates this setup and provides video output. Since there are no actual power circuits connected to a train on the FPGA board, it is only intended to give you a visual indication of how the output signals work in the real system. The following sections describe how the state machine should control each signal to operate the trains properly.

8.2 Train Direction Outputs (DA1-DA0, and DB1-DB0)

The direction for each train is controlled by four output signals (two for each train), DA (DA1-DA0) for train A, and DB (DB1-DB0) for train B[2]. When these signals indicate forward "01" for a particular train, a train will move counterclockwise (on track 4, the train moves toward the outer track). When the signals imply reverse "10", the train(s) will move clockwise. The "11" value is illegal and should not be used. When these signals are set to "00", a train will stop. (See Figure 8.2.)

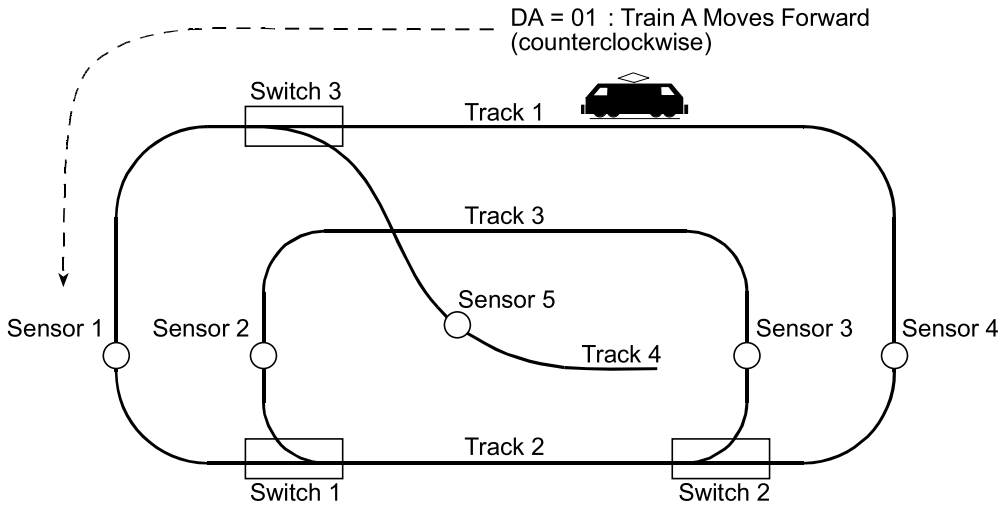


Figure 8.2 Controlling the train's motion with the train direction signals.

[2] For those familiar with earlier editions of this book, additional track power signals were required for power control relays. This new train problem is based on newer digital DCC model trains and it no longer needs the track power signals and relays, so they have been eliminated. The signals work exactly the same as the previous train setup, if you assume that track power supply A always runs train A and track power supply B always runs train B.

8.3 Switch Direction Outputs (SW1, SW2, and SW3)

Switch directions are controlled by asserting the SW1, SW2, and SW3 output signals either high (outside connected with inside track) or low (outside tracks connected). That is, anytime all of the switches are set to 1, the tracks are setup such that the outside tracks are connected to the inside tracks. (See Figure 8.3.)

If a train moves the wrong direction through an open switch it will derail. Be careful. If a train is at the point labeled "Track 1" in Figure 8.3 and is moving to the left, it will derail at Switch 3. To keep it from derailing, SW3 would need to be set to 0.

Also, note that Tracks 3 and 4 cross at an intersection and care must be taken to avoid a crash at this point.

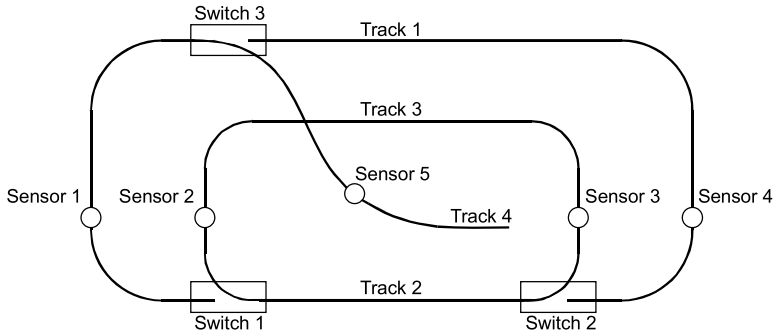


Figure 8.3 Track direction if all switches are asserted (SW1 = SW2 = SW3 = 1)

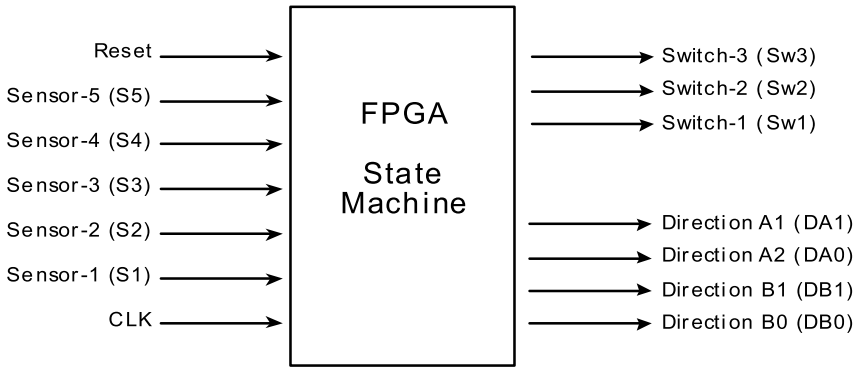
8.4 Train Sensor Input Signals (S1, S2, S3, S4, and S5)

The five train sensor input signals (S1, S2, S3, S4, and S5) go high when a train is near the sensor location. It should be noted that sensors (S1, S2, S3, S4, and S5) *do not go high for only one clock cycle*. In fact, the sensors fire continuously for *many* clock cycles per passage of a train. This means that if your design is testing the same sensor from one state to another, you must wait for the signal to change from high to low.

As an example, if you wanted to count how many times that a train passes Sensor 1, you can not just have an "IF S1 GOTO count-one state" followed by "IF S1 GOTO count-two state." You would need to have a state that sees S1=‘1’, then S1=‘0’, then S1=‘1’ again before you can be sure that it has passed S1 twice. If your state machine has two concurrent states that look for S1=’1’, the state machine will pass through both states in two consecutive clock cycles although the train will have passed S1 only once.

Another way would be to detect S1=’1’, then S4=’1’, then S1=’1’ if, in fact, the train was traversing the outside loop continuously. Either method will ensure that the train passed S1 twice.

The state machine's signal inputs and outputs have been summarized in the following figure:



Sensor (S1, S2, S3, S4, S5) = 1 Train Present

= 0 Train not Present

Switches (SW1, SW2, SW3) = 0 Connected to Outside Track

= 1 Connected to Inside Track

Train Direction (DA1-DA0) and (DB1-DB0) = 00 Stop

= 01 Forward (Counterclockwise)

= 10 Backward (Clockwise)

Figure 8.4 Train Control State Machine I/O Configuration.

8.5 An Example Controller Design

We will now examine a working example of a train controller state machine. For this controller, two trains run counterclockwise at various speeds and avoid collisions. One Train (A) runs on the outer track and the other train (B) runs on the inner track. Only one train at a time is allowed to occupy the common track.

Both an ASM chart and a classic state bubble diagram are illustrated in Figures 8.5 and 8.6 respectively. In the ASM chart, state names, ABout, Ain, Bin, Bstop, and Astop indicate the active and possible states. The rectangles contain

the active (High) outputs for the given state. Outputs not listed are always assumed to be inactive (Low).

The diamond shapes in the ASM chart indicate where the state machine tests the condition of the inputs (S1, S2, etc.). When two signals are shown in a diamond, they are both tested at the same time for the indicated values.

A state machine classic bubble diagram is shown in Figure 8.6. Both Figures 8.5 and 8.6 contain the same information. They are simply different styles of representing a state diagram. The track diagrams in Figure 8.7 show the states visually. In the state names, "in" and "out" refer to the state of track 2, the track that is common to both loops.

Description of States in Example State Machine

All States

- All signals that are not "Asserted" are zero and imply a logical result as described.

ABout: "Trains A and B Outside"

- DA0 Asserted: Train A is on the outside track and moving counterclockwise (forward).
- DB0 Asserted: Train B is on the inner track (not the common track) and also moving forward.
- Note that by NOT Asserting DA1, it is automatically zero -- same for DB1. Hence, the outputs are DA = “01” and DB = “01”.

Ain: "Train A moves to Common Track"

- Sensor 1 has fired either first or at the same time as Sensor 2.
- Either Train A is trying to move towards the common track, or
- Both trains are attempting to move towards the common track.
- Both trains are allowed to enter here; however, state Bstop will stop B if both have entered.
- DA0 Asserted: Train A is on the outside track and moving counterclockwise (forward).
- DB0 Asserted: Train B is on the inner track (not the common track) and also moving forward.

Bstop: "Train B stopped at S2 waiting for Train A to clear common track"

- DA0 Asserted: Train A is moving from the outside track to the common track.
- Train B has arrived at Sensor 2 and is stopped and waits until Sensor 4 fires.
- SW1 and SW2 are NOT Asserted to allow the outside track to connect to common track.

Bin: "Train B has reached Sensor 2 before Train A reaches Sensor 1"

- Train B is allowed to enter the common track. Train A is approaching Sensor 1.
- DA0 Asserted: Train A is on the outside track and moving counterclockwise (forward).
- DB0 Asserted: Train B is on the inner track moving towards the common track.
- SW1 Asserted: Switch 1 is set to let the inner track connect to the common track.
- SW2 Asserted: Switch 2 is set to let the inner track connect to the common track.

Astop: "Train A stopped at S1 waiting for Train B to clear the common track"

- DB0 Asserted: Train B is on the inner track moving towards the common track.
- SW1 and SW2 Asserted: Switches 1 and 2 are set to connect the inner track to the common track.

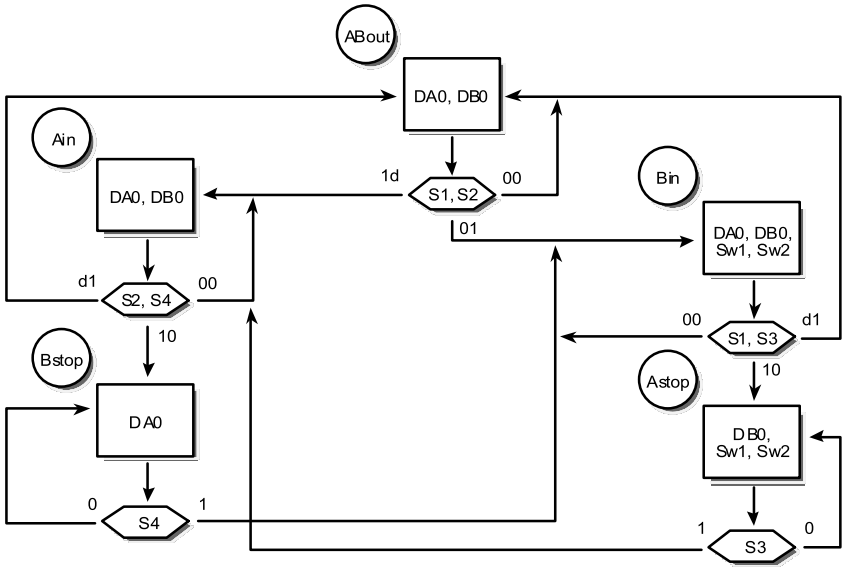


Figure 8.5 Example Train Controller ASM Chart.

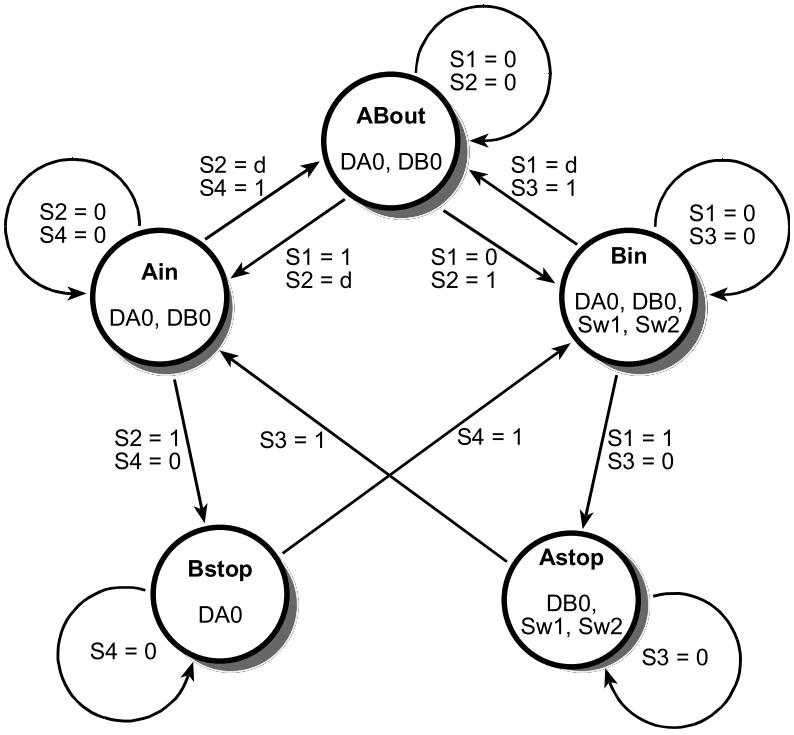


Figure 8.6 Example Train Controller State Diagram.

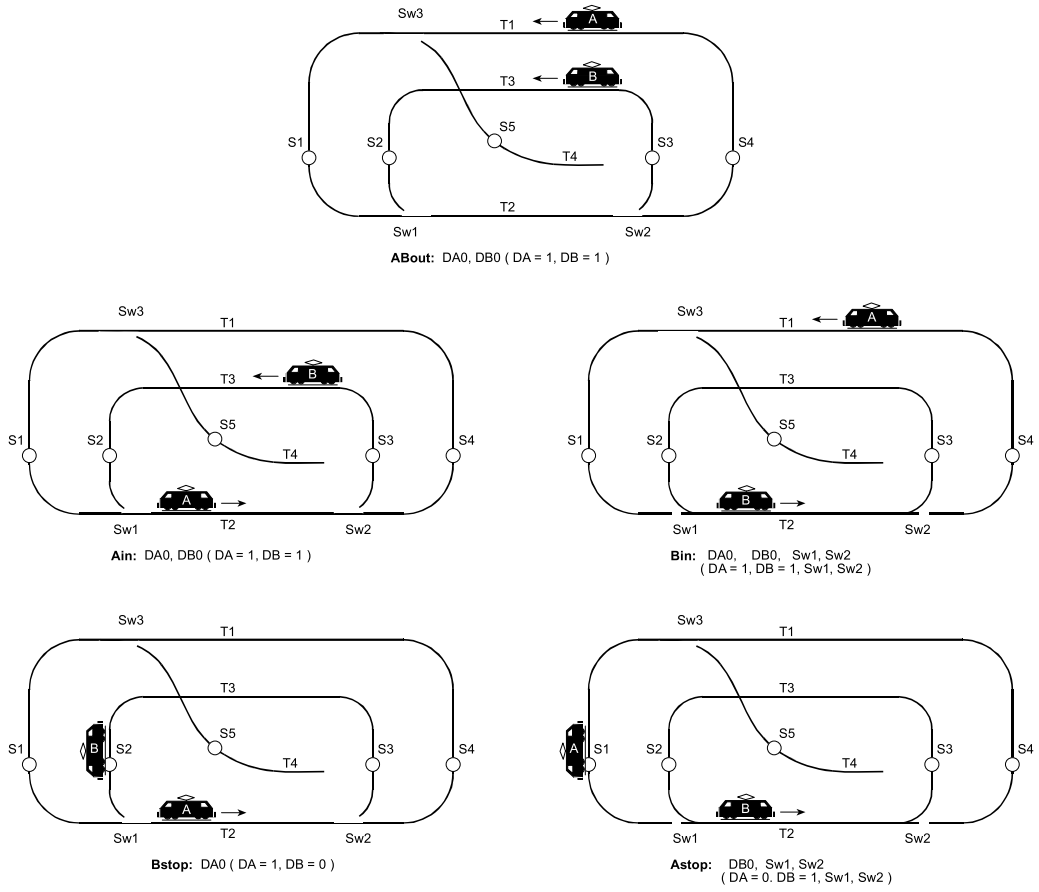


Figure 8.7 Working diagrams of train positions for each state.

Table 8.1 Outputs corresponding to states.

State	ABout	Ain	Astop	Bin	Bstop
Sw1	0	0	1	1	0
Sw2	0	0	1	1	0
Sw3	0	0	0	0	0
DA(1-0)	01	01	00	01	01
DB(1-0)	01	01	01	01	00

8.6 VHDL Based Example Controller Design

The corresponding VHDL code for the state machine in Figures 8.5 and 8.6 is shown below. A CASE statement based on the current state examines the inputs to select the next state. At each clock edge, the next state becomes the current state. WITH…SELECT statements at the end of the program specify the

outputs for each state. For additional VHDL help, see the help files in the Altera CAD tools or look at the VHDL examples in Chapter 6.

```
-- Example State machine to control trains-- File: Tcontrol.vhd
--
-- These libraries are required in all VHDL source files
LIBRARY IEEE;
USE  IEEE.STD_LOGIC_1164.ALL;
USE  IEEE.STD_LOGIC_ARITH.ALL;
USE  IEEE.STD_LOGIC_UNSIGNED.ALL;

                -- This section defines state machine inputs and outputs
                -- No modifications should be needed in this section
ENTITY Tcontrol IS
PORT(  reset, clock, sensor1, sensor2,
        sensor3, sensor4, sensor5       : IN      STD_LOGIC;
        switch1, switch2, switch3       : OUT     STD_LOGIC;
                -- dirA and dirB are 2-bit logic vectors(i.e. an array of 2 bits)
       dirA, dirB                       : OUT STD_LOGIC_VECTOR( 1 DOWNTO 0 ));
END Tcontrol;

                        -- This code describes how the state machine operates
                        -- This section will need changes for a different  state machine
ARCHITECTURE a OF Tcontrol IS

                        -- Define local signals (i.e. non input or output signals) here
    TYPE STATE_TYPE IS ( ABout, Ain, Bin, Astop, Bstop );
    SIGNAL state: STATE_TYPE;
    SIGNAL sensor12, sensor13, sensor24   : STD_LOGIC_VECTOR(1 DOWNTO 0);

BEGIN
                        -- This section describes how the state machine behaves
                        -- this process runs once every time reset or the clock changes
    PROCESS ( reset, clock )
    BEGIN
                        -- Reset to this state (i.e. asynchronous reset)
        IF reset = '1' THEN
                state <= ABout;
        ELSIF clock'EVENT AND clock = '1' THEN

                        -- clock'EVENT means value of clock just changed
                        --This section will execute once on each positive clock edge
                        --Signal assignments in this section will generate D flip-flops
                        -- Case statement to determine next state
                CASE state IS
                        WHEN ABout =>
                                -- This Case checks both sensor1 and sensor2 bits
                           CASE Sensor12 IS
                                -- Note: VHDL's use of double quote for bit vector versus
                                -- a single quote for only one bit!
                                WHEN "00" => state <= About;
                                WHEN "01" => state <= Bin;
```

```
                        WHEN "10" => state <= Ain;
                        WHEN "11" => state <= Ain;
                        -- Default case is always required
                        WHEN OTHERS => state <= ABout;
                    END CASE;

                WHEN Ain =>
                    CASE Sensor24 IS
                        WHEN "00" => state <= Ain;
                        WHEN "01" => state <= ABout;
                        WHEN "10" => state <= Bstop;
                        WHEN "11" => state <= ABout;
                        WHEN OTHERS => state <= ABout;
                    END CASE;

                WHEN Bin =>
                    CASE Sensor13 IS
                        WHEN "00" => state <= Bin;
                        WHEN "01" => state <= ABout;
                        WHEN "10" => state <= Astop;
                        WHEN "11" => state <= About;
                        WHEN OTHERS => state <= ABout;
                    END CASE;

                WHEN Astop =>
                    IF Sensor3 = '1' THEN
                        state <= Ain;
                    ELSE
                        state <= Astop;
                    END IF;

                WHEN Bstop =>
                    IF Sensor4 = '1' THEN
                        state <= Bin;
                    ELSE
                        state <= Bstop;
                    END IF;
            END CASE;
    END IF;
END PROCESS;
                    -- combine sensor bits for case statements above
                    -- "&" operator combines bits
sensor12 <= sensor1 & sensor2;
sensor13 <= sensor1 & sensor3;
sensor24 <= sensor2 & sensor4;

                    -- These outputs do not depend on the state
  Switch3 <= '0';
```

```
                    -- Outputs that depend on state, use state to select value
                    -- Be sure to specify every output for every state
                    -- values will not default to zero!
    WITH state SELECT
            Switch1   <=      '0'     WHEN ABout,
                              '0'     WHEN Ain,
                              '1'     WHEN Bin,
                              '1'     WHEN Astop,
                              '0'     WHEN Bstop;
    WITH state SELECT
            Switch2    <=     '0'     WHEN ABout,
                              '0'     WHEN Ain,
                              '1'     WHEN Bin,
                              '1'     WHEN Astop,
                              '0'      WHEN Bstop;
    WITH state SELECT
            DirA      <=      "01"    WHEN ABout,
                              "01"    WHEN Ain,
                              "01"    WHEN Bin,
                              "00"    WHEN Astop,
                              "01"    WHEN Bstop;
    WITH state SELECT
            DirB      <=      "01"    WHEN ABout,
                              "01"    WHEN Ain,
                              "01"    WHEN Bin,
                              "01"    WHEN Astop,
                              "00"    WHEN Bstop;
END a;
```

8.7 Verilog Based Example Controller Design

The corresponding Verilog code for the state machine in Figures 8.5 and 8.6 is shown below. A CASE statement based on the current state examines the inputs to select the next state. At each clock edge, the next state becomes the current state. A second CASE statement at the end of the program specifies the outputs for each state. For additional Verilog help, see the help files in the Altera CAD tools or look at the Verilog examples in Chapter 7.

```
                    // Example Verilog State machine to control trains
module Tcontrol (reset, clock, sensor1, sensor2, sensor3, sensor4, sensor5,
                    switch1, switch2, switch3, dirA, dirB);
                    // This section defines state machine inputs and outputs
                    // No modifications should be needed in this section
input reset, clock, sensor1, sensor2, sensor3, sensor4, sensor5;
output switch1, switch2, switch3;
output [1:0] dirA, dirB;
reg switch1, switch2;
                    // dirA and dirB are 2-bit logic vectors(i.e. an array of 2 bits)
reg [1:0] dirA, dirB;
reg [2:0] state;
```